

OS-Lab5

思考题

Thinking5.1

可能产生的问题：

- 映射到kseg0和内核中的数据或代码段发生冲突。
- 写入出现延迟与丢失，CPU发出的控制命令或数据滞留在cache中
- 外设寄存器的值可能会自发变化，这种变化不会被cache记录，使CPU读取错误数据。

设备差异：

- 串口设备
 - 依赖时许和单字节交互
 - 最明显的问题是状态轮询失败和字符丢失
- IDE磁盘
 - 涉及大块数据传输（配合DMA）
 - 缓存一致性：DMA直接操作物理内存，而CPU在cache中操作，如果数据还在cache没协会内存，DMA会读到旧数据；如果DMA把新数据写入内存而cache么更新，CPU读到错误数据。

Thinking5.2

在 user/include/fs.h 中：

```
#define BLOCK_SIZE PAGE_SIZE // #define PAGE_SIZE 4096
#define FILE_STRUCT_SIZE 256

#define FILE2BLK (BLOCK_SIZE / sizeof(struct File))
```

- 可看出一个磁盘块有16个文件控制块。
- 有1024个目录项，因此有 $16 \times 1024 = 16K$ 个文件。
- 单个文件最多占用1024个磁盘块， $1024 \times 4096 = 4MB$ 大小。

Thinking5.3

在 fs/serv.h 中：

```
/* Maximum disk size we can handle (1GB) */  
#define DISKMAX 0x40000000
```

可看出可以处理的最大磁盘大小为1GB.

Thinking5.4

在 user/include/fs.h 中：

```
#define BLOCK_SIZE PAGE_SIZE  
#define BLOCK_SIZE_BIT (BLOCK_SIZE * 8)  
#define FILE_STRUCT_SIZE 256  
#define FILE2BLK (BLOCK_SIZE / sizeof(struct File))  
  
struct File {  
    char f_name[MAXNAMELEN];  
    uint32_t f_size;  
    uint32_t f_type;  
    uint32_t f_direct[NDIRECT];  
    uint32_t f_indirect;  
  
    struct File *f_dir;  
    char f_pad[FILE_STRUCT_SIZE - MAXNAMELEN - (3 + NDIRECT) * 4 - sizeof(void *)];  
} __attribute__((aligned(4), packed));
```

其中展示了文件控制块的结构。

在 fs/serv.h 中：

```
#define PTE_DIRTY 0x0004
// 脏位

#define SECT_SIZE 512
#define SECT2BLK (BLOCK_SIZE / SECT_SIZE)
// 磁盘扇区大小和一个文件块包含多少个扇区

#define DISKMAP 0x10000000
#define DISKMAX 0x40000000
// 磁盘映射区的起始虚拟地址和结束虚拟地址
```

Thinking5.5

会共享。

测试代码如下：

```

#include <lib.h>

int main() {
    int r;
    int fdnum;
    char buf[512];
    int n;

    if ((r = open("/newmotd", O_RDWR)) < 0) {
        user_panic("wrong when open /newmotd: %d", r);
    }
    fdnum = r;
    debugf("fdnum == %d\n", fdnum); // 拿到fdnum号
    if ((n = read(fdnum, buf, 5)) < 0) {
        user_panic("wrong when read /newmotd: %d", r);
    } // 读5个字符
    debugf("read: %s\n", buf);
    struct Fd *fdd;
    fd_lookup(r, &fdd);
    debugf("fd == %x\n", fdd); // fdd的值
    debugf("now offset == %d\n", fdd->fd_offset); // 偏移量
    int id;
    if ((id = fork()) == 0) {
        if ((n = read(fdnum, buf, 5)) < 0) {
            user_panic("child: wrong when read /newmotd: %d", r);
        }
        debugf("child read: %s\n", buf);
        struct Fd *fdd;
        fd_lookup(r, &fdd);
        debugf("child: fd == %x\n", fdd);
        debugf("child: now offset == %d\n", fdd->fd_offset);
    }
    else {
        if ((n = read(fdnum, buf, 5)) < 0) {
            user_panic("father: wrong when read /newmotd: %d", r);
        }
        debugf("father read: %s\n", buf);
        struct Fd *fdd;
        fd_lookup(r, &fdd);
        debugf("father: fd == %x\n", fdd);
        debugf("father: now offset == %d\n", fdd->fd_offset);
    }
}

```

```
}  
}
```

运行结果:

```
init.c: mips_init() is called  
Memory size: 65536 KiB, number of pages: 16384  
to memory 80430000 for struct Pages.  
pmap.c: mips vm init success  
FS is running  
superblock is good  
read_bitmap is good  
fdnum == 0  
read: This  
fd == 5fc00000  
now offset == 5  
father read: is a  
father: fd == 5fc00000  
father: now offset == 10  
child read: diffe  
child: fd == 5fc00000  
child: now offset == 15  
[00000800] destroying 00000800  
[00000800] free env 00000800  
i am killed ...  
[00001802] destroying 00001802  
[00001802] free env 00001802  
i am killed ...  
panic at sched.c:45 (schedule): schedule: no runnable envs  
  
ra: 800260a0 sp: 803ffe80 Status: 00008000  
Cause: 00000420 EPC: 004002c0 BadVA: 7fd7f004  
curenv: NULL  
cur_pgdir: 83fd1000
```

Thinking5.6

```
struct File { // 文件控制块
    char f_name[MAXNAMELEN]; // filename
    uint32_t f_size; // 文件大小的字节数
    uint32_t f_type; // 文件类型
    uint32_t f_direct[NDIRECT]; // 直接索引，放着NDIRECT个磁盘块号
    uint32_t f_indirect; // 间接索引，指向磁盘块

    struct File *f_dir; // 指向所在的目录文件
    char f_pad[FILE_STRUCT_SIZE - MAXNAMELEN - (3 + NDIRECT) * 4 - sizeof(void *)]; // 字节对齐，
} __attribute__((aligned(4), packed));

struct Fd { // 用于描述文件操作
    u_int fd_dev_id; // 文件对应的设备id
    u_int fd_offset; // 文件当前读写的偏移量
    u_int fd_omode; // 文件的打开方式
};

struct Filefd { // 文件操作所需的整体信息
    struct Fd f_fd; // 文件的相关描述
    u_int f_fileid;
    struct File f_file; // 文件控制块
};
```

Thinking5.7

ENV_CREATE(user_env) 和 ENV_CREATE(fs_serv) 用于进程的启动。

ipc_send(fsreq) 和 ipc_send(dst_va) 代表IPC请求和IPC响应。

难点分析

lab5难点较多，一一展开过于繁杂，此处仅列出重难点。

- 磁盘IDE的读写操作： `ide_read` , `ide_write`
- 磁盘上的文件系统架构： `disk`与`block`的关系，`block`与`va`的关系，`block`与`file`的关系，`file`与文件的关系
- 用户发起文件系统操作： `file.c` 调用 `fsipc.c`
- `serv`实现文件系统操作

实验体会

好多，好难...

文件系统可以说是迄今为止内容量最大的lab，光是文件数量就多了一大堆，涉及函数更是多达上百个。只是照着指导书学习的话会漏掉许多细节，因此还是需要认真读一遍文件中的重要函数和重要宏。

好在exam和extra考的都比较简单。我觉得我的lab5学习甚至一直持续到考试过程中...extra中提供了一个完整的文件系统服务调用添加过程，看到这个提示后我才第一次完整梳理了文件系统服务调用过程。